

ASSIGNMENT # 3

SUBMITTED TO

MR. FARHAD M. RIAZ

SUBMITTED BY

TAYYABA RIAZ

ROLL No

2751

COURSE

PARALLEL & DIST. COMPUTING

BSCS, 8TH A, MORNING

QUESTION # 1

You are required to analyze - - - - - implementations.

1. CUDA vs CPU PERFORMANCE ANALYSIS

CPU (Sequential Execution)

- Executes instructions one after another
- Limited parallelism (usually 1-8 cores)
- Best for small datasets
- Lower overhead (no memory transfer)

GPU (CUDA Parallel Execution)

- Thousands of lightweight threads
- Each thread handles one element
- Best for large datasets
- Overhead of CPU → GPU memory transfer

2. SEQUENTIAL CPU IMPLEMENTATION

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{  
    int N = 1000000;
```

```
    float *A = (float *) malloc (N * sizeof (float));
```

```
    float *B = (float *) malloc (N * sizeof (float));
```

```
    float *C = (float *) malloc (N * sizeof (float));
```

```
    for (int i = 0; i < N; i++)
```

```
    {  
        A[i] = i * 1.0f;
```

```
        B[i] = i * 2.0f;
```

```
    }
```

```
    for (int i = 0; i < N; i++)
```

```
    {  
        C[i] = A[i] + B[i];
```

```
    }
```



```
printf("CPU vector Addition Completed\n");
```

```
free(A);
```

```
free(B);
```

```
free(C);
```

```
return 0;
```

```
}
```

CUDA GPU IMPLEMENTATION

```
#include <stdio.h>
```

```
#include <cuda_runtime.h>
```

```
-global- void vectorAdd(float *A, float *B, float *C, int N){
```

```
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    if (idx < N){
```

```
        C[idx] = A[idx] + B[idx];
```

```
    }
```

```
}
```

```
int main(){
```

```
    int N = 1000000;
```

```
    int size = N * sizeof(float);
```

```
    float *h_A, *h_B, *h_C;
```

```
    float *d_A, *d_B, *d_C;
```

```
    h_A = (float*) malloc(size);
```

```
    h_B = (float*) malloc(size);
```

```
    h_C = (float*) malloc(size);
```

```
    for(int i = 0; i < N; i++){
```

```
        h_A[i] = i * 1.0f;
```

```
        h_B[i] = i * 2.0f;
```

```
    }
```

```
    cudaMalloc((void**) &d_A, size);
```

```
    cudaMalloc((void**) &d_B, size);
```

```
    cudaMalloc((void**) &d_C, size);
```

```

    cudaMemcpy(d-A, h-A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d-B, h-B, size, cudaMemcpyHostToDevice);
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d-A, d-B, d-C, N);
    cudaMemcpy(h-C, d-C, size, cudaMemcpyDeviceToHost);
    printf("CUDA Vector Addition Completed");
    cudaFree(d-A);
    cudaFree(d-B);
    cudaFree(d-C);
    free(h-A);
    free(h-B);
    free(h-C);
    return 0;
}

```

3. PERFORMANCE COMPARISON

Implementation
CPU (Sequential)
GPU (CUDA)

Time ($N = 10^7$)
~ 0.30 - 0.50s
~ 0.01 - 0.03s

QUESTION # 2

Analyze the role of CUDA ----- on performance

ROLE OF CUDA MEMORY

GLOBAL VS SHARED MEMORY

- Inputs vectors A, B & output vector C are stored global memory
- Global memory is large but has high latency
- Shared memory is not used because each thread accesses unique data with no reuse.

MEMORY ACCESS PATTERNS

- Threads access contiguous memory locations
- Results in coalesced global memory access
- Improves memory bandwidth & reduces latency

IMPACT OF BLOCK & GRID DIMENSIONS

- Block size (e.g. 256 threads) improves occupancy & hides memory latency.
- Grid size ensures one thread per element.
- Proper configuration maximizes parallelism & performance

So, in summary, efficient memory access, coalesced access patterns, & optimal block-grid configuration ensure high CUDA performance, while shared memory is unnecessary for vector addition.

QUESTION # 3

Analyze potential - - - - - occupancy.

PERFORMANCE BOTTLENECKS IN CUDA

KERNEL LAUNCH OVERHEAD

- Each kernel launch introduces a fixed overhead
- Significant for small problem sizes
- Less noticeable for large-scale computations.

MEMORY TRANSFER LATENCY

- Data Transfer between host & device (CPU $\leftrightarrow$ GPU) is costly
- Can determine execution time if computation per element is small.
- Reduces overall GPU speedup for small inputs

THREAD DIVERGENCE & OCCUPANCY

Minimal thread divergence since all threads execute the same instructions.

Improper block size can reduce occupancy.

Low occupancy leads to poor latency hiding & reduced performance

QUESTION # 4

Consider a distributed - - - - - Byzantine).

1. POSSIBLE FAILURE TYPES

a) Node Failure

a compute node crashes or becomes unreachable.

b) Process Failure

One or more parallel processes terminate unexpectedly.

c) Network Failure

Message loss, delay, or network partition between nodes.

2. IMPACT OF FAILURES ON CORRECTNESS &

PERFORMANCE

Correctness:

- Missing processes can lead to incorrect or incomplete results
- Loss of synchronization causes inconsistent state

Performance:

- Increased execution time due to retries or recovery
- Load imbalance if some nodes stop working
- Possible complete application failure

3. CLASSIFICATION OF FAILURES

a) Fail-Stop Failure

- Component stops & does not produce incorrect results
- Easy to detect (e.g. node crash)

b) Omission Failure

- Messages or response are lost
- Caused by network or buffer overflow

c) Byzantine Failure

- Component behaves arbitrarily or maliciously
- Hardest to detect & handle

QUESTION # 5

Analyze & Compare ----- trade-offs

CHECKPOINTING vs REPLICATION

Checkpointing

- Periodically saves system/application state to stable storage
- On failure, system restarts from the last checkpoint

Replication

- Multiple copies of a process run on different node
- If ~~no~~ one fails another continues execution

COMPARISON

<u>Aspect</u>	<u>Checkpointing</u>	<u>Replication</u>
Resource overhead	Low (storage + I/O)	High (extra CPU, memory, nodes)
Recovery Time	High (rollback + restart)	Very low (immediate failover)
Performance Trade-off	Runtime overhead during checkpointing	Continuous overhead due to replicas

Active vs Passive Replication

Active Replication

- All replicas execute requests simultaneously
- Requires strong synchronization

Passive Replication

- One primary executes, backups stay idle
- Backups update state via checkpoints or logs

COMPARISON

<u>Aspect</u>	<u>Active Replication</u>	<u>Passive Replication</u>
Resource overhead	Very High	Moderate
Recovery Time	Minimal	Higher (leader switch)
Performance Trade-off	High synchronization cost	Lower runtime overhead

QUESTION # 6

Analyze the trade-off - - - - - acceptable

EFFECT OF FAULT TOLERANCE ON LATENCY & THROUGHPUT

Latency Increase

- Checkpointing, replication, & logging and extra steps
- Synchronization among replicas or saving state increases response time

Throughput Reduction

- Extra computations & communication for fault tolerance consume resources
- Fewer resources are available for normal workload, reducing overall throughput.

Trade-off

- Higher reliability usually comes at the cost of higher latency & lower throughput

- System design must balance fault tolerance vs performance requirements

SCENARIOS WHERE MINIMAL FAULT TOLERANCE

Is Acceptable

Non-Critical or batch applications

Example: large-scale data analysis where some jobs loss is tolerable

Short-lived processes

Jobs that complete quickly & can be restarted without significant impact.

Experimental or testing environments

Performance is prioritized over reliability.
